

Project Development Document

SP-202 — Agentic Software Debugger

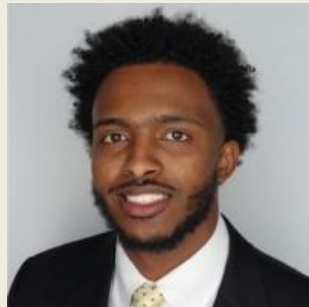
CS 4850 - Section 01 – Spring 2026

Sharon Perry

April 21, 2026

Team Members:

Nahum Tadesse, Vignesh Reddy Vanteru



Nahum Tadesse
Team Leader
Software Development &
Testing



Vignesh Reddy Vanteru
Software Development &
Documentation

STATS AND STATUS

- **Lines of Code (LOC):** 795
- **Components/Tools:** System Controller, Detection Agent, Fix Agent, Documentation Agent, Flake8, Pylint, Google ADK
 - **Hours Estimate:** 243
 - **Hours Actual:** 200
- **Status:** Project is 100% complete and working as designed

TABLE OF CONTENTS

TABLE OF CONTENTS	2
1. INTRODUCTION	3
Collaboration Tools:	3
Abstract:	4
Platform:	4
Project Website:	4
Deliverables:	4
Group Meeting Schedule:	5
2. CHALLENGES, SOLUTIONS, AND ASSUMPTIONS	5
Challenges:	5
Solutions:	6
Assumptions:	6
3. REQUIREMENTS	6
Functional Requirements:	6
Non-Functional Requirements:	7
4. ANALYSIS / DESIGN	7
System Architecture:	7
System Integration & Workflow Overview	8
Detection Agent	8
Fix Agent	8
Documentation Agent	8
Connection Overview and Integration Logic	8
Prerequisites and Installation	9
Verification	9
5. DEVELOPMENT	9
Development Documentation:	9
Database Connection:	10
Project Setup:	10
6. TEST PLAN (STP)	10
Test Objectives	10
Scope of Testing	10
In-Scope:	10
Out-of-Scope:	11
Tests Cases	11
TC-01 — Syntax Error Detection	11
TC-02 — Multiple Error Fix	11
TC-03 — Clean Code	11
TC-04 — Documentation Accuracy	11
TC-05 — Corrected Code Verification	11
Test Prodecures	12
TC-01 — Syntax Error Detection	12
TC-02 — Multiple Error Fix	12
TC-03 — Clean Code	12
TC-04 — Documentation Accuracy	12
TC-05 — Corrected Code Verification	12
Test Environment	12
Test Data	13
Software Test Report (STR)	13
VERSION CONTROL	13
CONCLUSION / SUMMARY	14

1. INTRODUCTION

The Agentic Software Debugger is a Python-based multi-agent system designed to automatically analyze, fix, and document programming errors in user-submitted code. The system consists of three coordinating AI agents: a Bug Detection Agent, a Bug Fixing Agent, and a Documentation Agent, orchestrated by a central System Controller. The pipeline follows a structured Detect, Fix, Document workflow, where each agent receives structured JSON output from the previous stage. The system uses Google ADK (Gemini API) for AI-powered agent behavior, alongside Flake8 and Pylint for static analysis. A loop-based refinement process allows the Fix Agent to iteratively attempt corrections until errors are resolved or a maximum iteration limit is reached. The system is designed to never overwrite user code without explicit confirmation, preserving user control throughout the debugging process. Target users are entry-level to intermediate programmers who need detailed explanations of errors and proposed corrections.

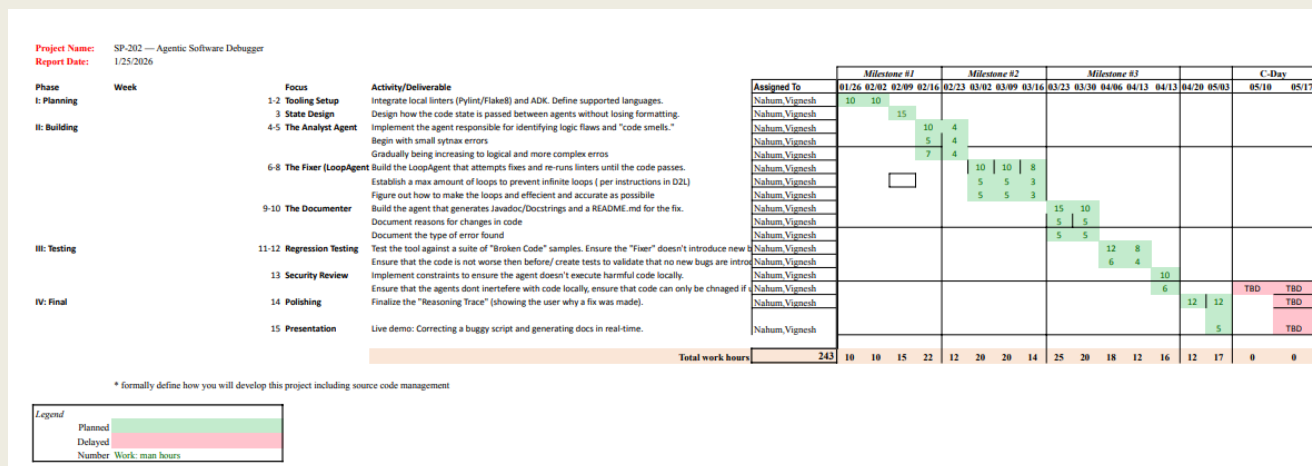
Collaboration Tools:

Communication — Teams, Cellphones (Call/Text)

Collaboration — Teams, Cellphones (Call/Text)

Connecting with Project Owner & Advisor — Teams, Outlook (Email)

Gnatt Chart:



Abstract:

Our goal with the Agentic Software Debugger is to essentially create 3 AI Agents that will work together. The first AI Agent is responsible for finding potential bugs in the code, our second AI agent will try to fix these errors, while the third AI agent will create documentation. We will be utilizing python and Google-ADK.

Platform:

Per the instructions for our project will use

- Google-ADK
- Python or Java
- A local environment for linter scripts ex. Pylint
- We will also use resources in including but not limited to the resources given on the project instructions
 - Google Colabs and ADK Concepts links in the instructions

Project Website:

[Link to our site](#)

Our project website is now live. It showcases all work completed throughout the semester, including our team, final report, and presentation.

Deliverables:

We will follow the 15 week Implementation Schedule already given to us.

Phase	Week	Focus	Activity/Deliverable
I: Planning	1-2	Tooling Setup	Integrate local linters (Pylint/Flake8) and ADK. Define supported languages.
	3	State Design	Design how the code state is passed between agents without losing formatting.
II: Building	4-5	The Analyst Agent	Implement the agent responsible for identifying logic flaws and "code smells."

	6-8	The Fixer (LoopAgent)	Build the LoopAgent that attempts fixes and re-runs linters until the code passes.
	9-10	The Documenter	Build the agent that generates Javadoc/Docstrings and a README.md for the fix.
III: Testing	11-12	Regression Testing	Test the tool against a suite of "Broken Code" samples. Ensure the "Fixer" doesn't introduce new bugs.
	13	Security Review	Implement constraints to ensure the agent doesn't execute harmful code locally.
IV: Final	14	Polishing	Finalize the "Reasoning Trace" (showing the user <i>why</i> a fix was made).
	15	Presentation	Live demo: Correcting a buggy script and generating docs in real-time.

* This is schedule came from the class D2L under Table of Contents>Projects Spring 2026>Course>SP-202-Agentic Software Debugger

Group Meeting Schedule:

We had our meetings t in person on Tuesday or Thursday weekly either before class or after class to discuss anything about the project. If we did not meet in person on those days, we scheduled a weekly online meeting via Teams on Fridays at 3pm.

2. CHALLENGES, SOLUTIONS, AND ASSUMPTIONS

Challenges:

- Coordinating three independent agents in a reliable sequential pipeline without data loss between stages
- Google ADK quotas and when Google ADK was down
- Handling poorly formatted or malformed code input without crashing the system
- Avoiding duplicate error reporting when multiple tools detect the same issue

Solutions:

- Implemented a shared structured JSON format as the communication contract between all agents, ensuring consistent data flow
- Added a configurable maximum iteration limit to the refinement loop, with a clear stopping condition when no errors remain
- Used Python AST parsing as a first-pass safety check before invoking Flake8/Pylint via subprocess calls
- Designed each agent as an independent, modular component so it can be tested and updated individually
- Fallback behavior implemented when the Fix Agent cannot produce a valid correction

Assumptions:

- Users provide source code as text input via the command line
- A valid Python environment is available on the user's machine
- Google SDK is correctly installed and authenticated with a valid API key (GOOGLE_API_KEY or GEMINI_API_KEY)
- Users will review and confirm proposed changes before they are applied
- The system runs in a local development environment; no special hardware is required

3. REQUIREMENTS

Functional Requirements:

- The system shall contain three agents (Detection, Fix, Documentation) that coordinate sequentially
- The Detection Agent shall identify syntax errors, logical inconsistencies, and structural problems in submitted code
- The Detection Agent shall return results in structured JSON format with enough detail for the Fix Agent to act
- The Fix Agent shall read from the Detection Agent's JSON output and attempt corrections while preserving the original code's intent
- The Fix Agent shall operate in a refinement loop, re-evaluating code after each fix attempt until errors are resolved or the iteration limit is reached

- The Documentation Agent shall generate a human-readable report summarizing all detected errors and changes made; it shall not modify any code
- The System Controller shall coordinate all three agents and maintain the JSON data structure throughout the pipeline
- The system shall require user confirmation before applying any changes to code or files
- The system shall output: proposed corrected code, a structured JSON debugging summary, and a human-readable documentation report

Non-Functional Requirements:

- Security: User code shall not be stored indefinitely; unauthorized code execution shall be prevented; input shall be sanitized
- Capacity: The system shall handle large code files with a configurable size limit; the refinement loop shall have a configurable iteration limit
- Usability: Output documentation shall be clear and readable; JSON output shall be well-structured; error messages shall be meaningful
- Reliability: The system shall have no infinite loops; it shall gracefully handle invalid input and recover from agent failures
- Portability: The system shall run on Windows, macOS, and Linux with Python 3.10+

4. ANALYSIS / DESIGN

System Architecture:

- The system follows a multi-agent pipeline architecture with four main components: System Controller, Detection Agent, Fix Agent, and Documentation Agent
- The System Controller (`system_controller.py`) serves as the central coordinator, managing execution order and passing data between agents
- Agents operate independently and in series, each consumes the output of the previous stage via a shared JSON structure
- The JSON structure contains: detected errors (type, location, description), current code state, iteration count, and fix history
- Google ADK's `SequentialAgent` and `LoopAgent` patterns are used to coordinate the pipeline

System Integration & Workflow Overview

The system is designed using a multi-agent architecture composed of three main components: the Detection Agent, Fix Agent, and Documentation Agent, all coordinated by a central System Controller that manages execution flow. Each component operates independently and is responsible for a specific task in the debugging process, with communication handled through a shared structured JSON format to ensure consistency, traceability, and ease of integration. The system functions as a multi-component pipeline where the System Controller passes data sequentially between agents, while also supporting a loop-based refinement process that repeatedly detects and attempts to fix errors. The Detection Agent analyzes code using Python AST parsing for syntax errors, Flake8 for static analysis, and Pylint for deeper error detection, returning results in a consistent JSON structure.

Detection Agent

The Detection Agent is responsible for identifying errors in the input code and returning them in a structured JSON format. It was developed by incorporating AST parsing for syntax error detection and integrating Flake8 and Pylint for additional analysis, with all results formatted into a consistent JSON structure.

Fix Agent

The Fix Agent is designed to attempt corrections based on the errors identified in the code and operates within the system's loop-based refinement process. It is currently implemented as a placeholder that returns the original code, but it is structured to receive both code and error reports and is prepared for future rule-based or AI-based fixing logic.

Documentation Agent

The Documentation Agent generates a summary of the debugging process and explains the results to the user. It is currently implemented as a simple placeholder that takes the final code and error data as input, with a design that allows for future expansion.

Connection Overview and Integration Logic

The system does not use a database and operates entirely on user-provided code input. It utilizes external tools such as Flake8 and Pylint, along with Google ADK for building AI agents. Integration is achieved by running Flake8 and Pylint through subprocess calls, writing the code to a temporary file for analysis, capturing the output, converting it into JSON format, and passing this structured data between system components.

Future development includes integrating Google ADK to enhance or replace agents with AI capabilities.

Prerequisites and Installation

The system requires Python, an IDE such as PyCharm, and Python packages including flake8 and pylint installed using pip. Setup involves creating or downloading the project files, opening them in an IDE, creating and activating a virtual environment, installing required packages, and ensuring all files are correctly organized before running the program through main.py.

Verification

To verify the system, the program is run using main.py, prompting the user to input code and type END to complete the input. After entering a test code snippet, the system should display the input or corrected code, a detection report in JSON format, and a documentation summary. Successful execution without errors and proper output confirms correct setup, while code containing errors should result in identified issues within the detection report.

5. DEVELOPMENT

Development Documentation:

- The system is written entirely in Python and structured into five directories: agents/, controller/, models/, utils/, and tests/. The main.py serves as the entry point; it accepts multi-line code input from the user (terminated by typing END) and calls run_debugger() from the System Controller. The System Controller (controller/system_controller.py) orchestrates all agents and implements the refinement loop. The Detection Agent uses Python's built-in ast module along with subprocess-based calls to Flake8 and Pylint. The Fix Agent integrates with the Google Gemini API (google-generativeai / google-adk packages) to generate corrected code. The Documentation Agent produces a structured plain-text summary of the debugging session, with a Gemini-powered AI version and a deterministic fallback. Supporting utilities are housed in utils/ (e.g., json_utils.py,

debug_report.py) for JSON formatting and report generation. Data models are defined in the models/ directory to standardize the JSON schema used between agents

Database Connection:

- Not applicable —this project does not use a database. All state is maintained in memory during a single debugging session via the shared JSON structure.

Project Setup:

- Prerequisites: Python 3.10+, PyCharm or VS Code, pip
- Clone the repository from:
<https://github.com/NahumTadesse/AgenticAIDebugger>
- Create and activate a Python virtual environment (venv)
- Install required packages: pip install flake8 pylint google-generativeai google-adk
- Set your API key as an environment variable: GOOGLE_API_KEY or GEMINI_API_KEY
- Run the program with: python main.py
- When prompted, paste your code and type END on a new line to submit

6. TEST PLAN (STP)

Test Objectives

- Verify the System Controller correctly coordinates all three agents in sequence
- Validate the Detection Agent accurately identifies syntax errors using AST, Flake8, and Pylint
- Confirm agents communicate correctly via the shared JSON structure
- Ensure the Fix Agent receives error reports and returns code output
- Verify the Documentation Agent generates a summary after each debugging cycle
- Confirm the loop-based refinement process executes without crashing

Scope of Testing

In-Scope:

- Detection Agent — AST, Flake8, and Pylint error detection
- Fix Agent — Gemini API integration and fallback behavior
- Documentation Agent — AI report and deterministic fallback
- System Controller — pipeline execution and loop refinement logic
- Google ADK — SequentialAgent and LoopAgent coordination

- JSON communication structure between all agents
- Supporting modules — `debug_report.py` and `json_utils.py`

Out-of-Scope:

- Web-based interface (not yet implemented)
- Performance, load, and security testing

Tests Cases

TC-01 — Syntax Error Detection

- Step 1: Run `agent.py`
- Step 2: Submit code containing a syntax error
- Expected: Corrected code and proper documentation confirming the syntax error detected

TC-02 — Multiple Error Fix

- Step 1: Run `agent.py`
- Step 2: Submit code containing multiple errors
- Expected: Corrected code and proper documentation of all changes made

TC-03 — Clean Code

- Step 1: Run `agent.py`
- Step 2: Submit valid error-free Python code
- Expected: Corrected code and proper documentation confirming no changes were needed

TC-04 — Documentation Accuracy

- Step 1: Run `agent.py`
- Step 2: Submit code containing errors
- Expected: Documentation report is correctly generated and accurately reflects the debugging session

TC-05 — Corrected Code Verification

- Step 1: Run `agent.py`
- Step 2: Submit code containing errors
- Step 3: Take the outputted corrected code and run it
- Expected: Corrected code runs successfully without errors

Test Prodecures

TC-01 — Syntax Error Detection

- Step 1: Run agent.py
- Step 2: Submit code containing a syntax error
- Expected: Corrected code and proper documentation confirming the syntax error detected

TC-02 — Multiple Error Fix

- Step 1: Run agent.py
- Step 2: Submit code containing multiple errors
- Expected: Corrected code and proper documentation of all changes made

TC-03 — Clean Code

- Step 1: Run agent.py
- Step 2: Submit valid error-free Python code
- Expected: Corrected code and proper documentation confirming no changes were needed

TC-04 — Documentation Accuracy

- Step 1: Run agent.py
- Step 2: Submit code containing errors
- Expected: Documentation report is correctly generated and accurately reflects the debugging session

TC-05 — Corrected Code Verification

- Step 1: Run agent.py
- Step 2: Submit code containing errors
- Step 3: Take the outputted corrected code and run it
- Expected: Corrected code runs successfully without errors

Test Environment

- Operating System: Windows 10/11, macOS, or Linux
- Python Version: 3.10+
- IDE: PyCharm or VS Code
- Virtual Environment: venv with all packages installed
- Required Packages:
 - flake8
 - pylint

- google-generativeai
- google-adk
- API: Google Gemini via GOOGLE_API_KEY or GEMINI_API_KEY environment variable
- Network: Internet access required for Gemini API calls

Test Data

- TC-01 — Python code containing a syntax error
- TC-02 — Python code containing multiple errors
- TC-03 — Valid error-free Python code
- TC-04 — Python code containing errors to verify documentation output
- TC-05 — Python code containing errors to verify corrected output runs successfully

Software Test Report (STR)

Requirement	Pass	Fail	Severity
TC-01 — Syntax error detected and documented	Pass		Critical
TC-02 — Multiple errors fixed and documented	Pass		Critical
TC-03 — Clean code confirmed and documented	Pass		Moderate
TC-04 — Documentation report accurately reflects debugging session	Pass		Moderate
TC-05 — Corrected code runs successfully without errors	Pass		Critical

VERSION CONTROL

- The project uses **GitHub** for version control, hosted at:
<https://github.com/NahumTadesse/AgenticAIDebugger>
- A live project website detailing the architecture, functionality, and demo is available at:
https://viggy364.github.io/agentcy_debugger_website/
- A dedicated project GitHub account was created (separate from personal accounts) per the team's version control plan
- The repository is organized with clear directory structure: agents/, controller/, models/, utils/, tests/, and main.py at the root
- All source code changes were committed and tracked through the master branch

- The team used Git to manage collaboration between Nahum Tadesse and Vignesh Reddy Vanteru throughout development
- Team communication and coordination were handled via Microsoft Teams and direct messaging alongside GitHub commits

▸ AgenticAIDebugger/
▸ .adk/
▸ artifacts/
▸ adk_app/
▸ .adk/
· <code>__init__.py</code>
· <code>agent.py</code>
▸ agents/
▸ .adk/
· <code>__init__.py</code>
· <code>__init__.py</code>
· <code>documentation_agent.py</code>
· <code>error_detect_agent.py</code>
· <code>error_fix_agent.py</code>
▸ controller/
· <code>__init__.py</code>
· <code>system_controller.py</code>
▸ models/
· <code>debug_report.py</code>
▸ tests/
· <code>sample_bug.py</code>
▸ utils/
· <code>.env</code>
· <code>main.py</code>

CONCLUSION / SUMMARY

- The Agentic Software Debugger successfully implements a fully functional multi-agent debugging pipeline in Python using Google ADK and the Gemini API. All three core agents (Detection, Fix, and Documentation) are implemented and working within the Detect, Fix, Document pipeline. The System Controller correctly coordinates agent execution, maintains the shared JSON state, and manages the loop-based refinement process. The system demonstrates key software engineering principles including modularity, separation of concerns, safe user interaction (no auto-overwrite), and configurable stopping conditions. All 5 test cases passed, confirming the system handles syntax errors, multiple errors, clean code, documentation accuracy, and verified corrected output
- **Current Limitations:**
 - The Fix Agent's correction capability depends on Gemini API availability and response quality
 - Some errors may be reported in duplicate across detection tools (AST, Flake8, Pylint)
 - No graphical or web-based interface is implemented
- **Future Improvements:**
 - More advanced rule-based and AI-powered fix logic
 - Expanded support for additional programming languages
 - A web-based or IDE-integrated interface (e.g., VS Code extension)
 - Enhanced documentation output with reasoning traces
 - Deeper Google ADK integration for more autonomous agent behavior